

calculate_postTARE_ts_aws_peak_demand

April 15, 2026

Post-TARE Model Run: Timeseries Peak Load Analysis (AWS / BuildStockQuery)

Author: Jordan M. Joseph, PhD — Carnegie Mellon University

Queries the AWS-hosted ResStock EUSS 2022.1.1 timeseries database via BuildStockQuery to compute **county-level peak load changes** under two adoption scenarios: - **(a) 100% adoption counterfactual** — all filtered building IDs adopt (upper bound) - **(b) Economically-constrained adoption** — only Tier 1 + Tier 2 adopters (TARE output)

Prerequisite: Steps 0–0c must be run first (TARE data loaded, MP selected).

Dataset scope: ResStock 2022.1.1 (EUSS, AMY2018). ResStock 2025.1 is future work.

Primary test case: Allegheny County, PA (FIPS 42003) — validate here before scaling.

See methodology notes in `peak_load_methodology.md`.

Step 0: Imports and Configuration

```
[1]: import os
import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

from config import PROJECT_ROOT
from cmu_tare_model.constants import (
    ALLOWED_HOUSING_TYPES,
    VALID_MENU_MPS,
    VERBOSE,
    REMDB_COST_SCENARIO_KEYS,
    RCM_MODELS,
    PRIVATE_DISCOUNT_RATE_SHORT_KEYS,
)

from cmu_tare_model.utils.column_names import (
    create_npv_col,
    create_capital_col,
```

```

)

from cmu_tare_model.utils.load_exported_results_to_df import load_measure_package_data

from cmu_tare_model.adoption_kpis.kpi_functions import (
    mp_to_upgrade,
    load_euss_baseline,
    load_euss_upgrade,
    calculate_price_ratios,
    compute_thermal_cop_by_state,
    compute_spark_gap_metrics,
    compute_scenario_demand,
    aggregate_demand_by_state,
    FUEL_PRICES_PATH,
    SHAPEFILE_PATH,
    HEATING_FUEL_COLS,
    HP_BACKUP_ELEC_COL,
    HP_FANS_PUMPS_COL,
)

from cmu_tare_model.adoption_kpis.visualize_geospatial_data import (
    prepare_state_geodataframe,
    create_choropleth_map,
)

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 60)

print(" Imports loaded")

```

Project root directory: c:\users\jorda\desktop\projects\cmu-tare-model
Imports loaded

Step 0b: Measure Package Selection

```

[2]: SELECTABLE_MPS = [mp for mp in VALID_MENU_MPS if mp != 0]

try:
    _ = input_measure_package
    batch_mode = True
    selected_mps = [int(input_measure_package)]
    print(f"BATCH MODE: Running for MP{selected_mps[0]}")
except NameError:
    batch_mode = False
    print(f"Available measure packages: {SELECTABLE_MPS}")
    mp_input = input("Enter MP numbers (comma-separated, or 'all'): ").strip()
    if mp_input.lower() == 'all':

```

```

        selected_mps = SELECTABLE_MPS
    else:
        selected_mps = [int(x.strip()) for x in mp_input.split(',') if x.
↪strip().isdigit()]
        selected_mps = [mp for mp in selected_mps if mp in SELECTABLE_MPS]
    if not selected_mps:
        selected_mps = [4]
        print("No valid MPs selected. Defaulting to MP4.")

print(f"\nSelected measure packages: {selected_mps}")

```

Available measure packages: [3, 4]

Selected measure packages: [3]

Step 0c: Load TARE Model Data (Measure Packages 3, 4)

Load pre-computed TARE model outputs for the selected measure packages. Required for Step 4d (Private NPV extraction).

If the top-section data loading cells have already been run, this will reuse DATAFRAMES_BY_MP.

```

[3]: # =====
# STEP 0c: LOAD TARE MODEL DATA (for NPV extraction)
# =====
# Check if DATAFRAMES_BY_MP was already loaded by the top-section cells.
# If not, prompt for the output folder and load for selected MPs.

try:
    _ = DATAFRAMES_BY_MP
    print(f"DATAFRAMES_BY_MP already loaded: {list(DATAFRAMES_BY_MP.keys())}")
except NameError:
    print("DATAFRAMES_BY_MP not found - loading TARE model outputs...")

    # Check if output_folder_path is already defined (from top section)
    try:
        _ = output_folder_path
        print(f"    Using existing output_folder_path: {output_folder_path}")
    except NameError:
        output_folder_path = os.path.join(PROJECT_ROOT, "cmu_tare_model",
↪"output_results")
        location_id = input("Enter location ID (e.g., 'National' or 'PA'): ").
↪strip()
        model_run_date_time = input("Enter model run timestamp
↪(YYYY-MM-DD_HH-MM): ").strip()
        print(f"    output_folder_path: {output_folder_path}")
        print(f"    location_id: {location_id}")

```

```

print(f"  model_run_date_time: {model_run_date_time}")

DATAFRAMES_BY_MP = {}
for mp in selected_mps:
    DATAFRAMES_BY_MP[mp] = load_measure_package_data(
        mp, output_folder_path, location_id, model_run_date_time
    )

print(f"\n Loaded TARE data for MPs: {list(DATAFRAMES_BY_MP.keys())}")

```

```

DATAFRAMES_BY_MP not found - loading TARE model outputs...
  output_folder_path: c:\users\jorda\desktop\projects\cmu-tare-
model\cmu_tare_model\output_results
  location_id: National
  model_run_date_time: 2026-04-10_00-05
Loading MP3 data...
  fixed_base:
MP3 loading complete!

```

Loaded TARE data for MPs: [3]

Step 1: BuildStockQuery Installation and AWS Configuration

Installs BuildStockQuery (NREL SWR-23-58) and verifies AWS credentials are configured. BuildStockQuery connects to the OEDI data lake via AWS Athena and returns results as pandas DataFrames.

0.0.1 One-time setup (do once per machine)

1. Install packages (in the cmu-tare-model conda environment):

```

conda activate cmu-tare-model
pip install boto3
pip install git+https://github.com/NREL/buildstock-query.git

```

2. Create an AWS IAM Access Key: 1. Sign in to the [AWS Console](#) 2. Go to **IAM** → **Users** → (your user) → **Security credentials** 3. Click **Create access key** 4. Select use case: **Command Line Interface (CLI)** 5. Check the confirmation box and click **Next** → **Create access key** 6. **Copy both values immediately** — the Secret Access Key is only shown once: - Access Key ID (20 chars, starts with AKIA) - Secret Access Key (40-char random string)

3. Configure the AWS CLI:

```
aws configure
```

Enter the following when prompted: | Prompt | Value | |——|——| | AWS Access Key ID | AKIA... (from step 2) | | AWS Secret Access Key | (40-char string from step 2) | | Default region name | us-west-2 | | Default output format | json |

4. Verify credentials work:

```
aws sts get-caller-identity
```

You should see your Account ID and ARN printed — no errors.

0.0.2 Required IAM permissions

The IAM user needs these managed policies (or equivalent): - **AmazonAthenaFullAccess** — query the OEDI data lake - **AmazonS3ReadOnlyAccess** — read Athena query results from S3

0.0.3 References

- BuildStockQuery docs: <https://github.com/NREL/buildstock-query/wiki>
- BuildStockQuery examples: https://github.com/NREL/buildstock-query/tree/main/example_usage
- OEDI S3 browser: https://data.openei.org/s3_viewer?bucket=oedi-data-lake&prefix=nrel-pds-building-stock%2F
- AWS CLI install: <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

```
[5]: # CONTEXT: Setting up BuildStockQuery for ResStock EUSS 2022.1.1 timeseries
    ↪queries
# via AWS Athena on the OEDI data lake. Python 3.11, conda env: cmu-tare-model.
#
# TASK:
# 1. Check if buildstock_query is importable; if not, print install
    ↪instructions.
# 2. Check AWS credentials are configured (boto3 STS get_caller_identity).
# 3. Print the confirmed AWS region and identity as a sanity check.
# 4. Import ResStockQuery (or equivalent BuildStockQuery entry point).
#
# INPUTS: None (environment check only)
# OUTPUTS: Printed confirmation. `bsq_available` (bool). `ResStockQuery` class
    ↪imported.
# CONSTRAINTS: Type hints on any helper functions. Fail fast with a clear error
    ↪message
#
    if AWS credentials are missing. Do not hardcode any credentials.

# 1. Check BuildStockQuery availability
bsq_available: bool = False
BuildStockQuery = None

try:
    from buildstock_query import BuildStockQuery # type: ignore[import-untyped]
    bsq_available = True
    print(f" buildstock_query imported (BuildStockQuery class:
    ↪{BuildStockQuery})")
except ImportError:
```

```

print(
    " buildstock_query is NOT installed.\n"
    " Install with:\n"
    "     pip install git+https://github.com/NREL/buildstock-query\n"
    " Or for full features:\n"
    "     pip install git+https://github.com/NREL/
↪buildstock-query#egg=buildstock-query[full]"
)

# 2. Check AWS credentials
aws_identity: dict | None = None
aws_region: str | None = None

if bsq_available:
    import boto3
    from botocore.exceptions import (
        NoCredentialsError,
        ClientError,
        TokenRetrievalError,
    )

    session = boto3.session.Session()
    aws_region = session.region_name

    try:
        sts = session.client("sts")
        aws_identity = sts.get_caller_identity()
        print(f" AWS credentials valid")
        print(f" Account : {aws_identity['Account']}")
        print(f" ARN      : {aws_identity['Arn']}")
        print(f" Region  : {aws_region}")
    except NoCredentialsError:
        raise RuntimeError(
            "AWS credentials not found. Run `aws configure` or set "
            "AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY environment variables."
        )
    except TokenRetrievalError as e:
        raise RuntimeError(
            f"AWS SSO token retrieval failed - run `aws sso login` first.\n ↪
↪{e}"
        )
    except ClientError as e:
        raise RuntimeError(f"AWS STS call failed: {e}")
else:
    print(" Skipping AWS credential check (buildstock_query not installed).")

print("\n Step 1 COMPLETE")

```

```
INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials
```

```
buildstock_query imported (BuildStockQuery class: <class
'buildstock_query.main.BuildStockQuery'>)
AWS credentials valid
Account : 069765036196
ARN      : arn:aws:iam::069765036196:user/jordanjo@alumni.cmu.edu
Region   : us-west-2
```

Step 1 COMPLETE

[]:

Step 2: OEDI Table Discovery — Confirm EUSS 2022.1.1 Schema

Before writing any queries, we need to confirm the exact Athena database name, table names, and column schema for ResStock EUSS 2022.1.1 timeseries data.

Key unknowns to resolve in this step: - What is the Athena database name for EUSS 2022.1.1? - What are the timeseries table names (baseline = MP0, upgrade = MP3/MP4)? - What county-level geography columns exist? (FIPS code? GISJOIN? county name string?) - What is the exact column name and format for `bldg_id` in the timeseries table? - What are the electricity end-use column names for total consumption per hour? - What is the `timestamp` column name and format (ISO string? epoch integer? hour index 1–8760?)

OEDI S3 path (confirm via browser): `s3://oedi-data-lake/nrel-pds-building-stock/end-use-load-pr`

Expected output of this step: A printed schema summary that confirms all column names needed for Steps 3–6.

```
[32]: # CONTEXT: We need to discover the Athena schema for ResStock EUSS 2022.1.1
      ↪before
      # querying timeseries data. The data is on the OEDI data lake (AWS Athena).
      # BuildStockQuery or direct boto3/Athena calls can be used to inspect table
      ↪schemas.
      #
      # TASK:
      # 1. Initialize a BuildStockQuery ResStockQuery object pointing to EUSS 2022.
      ↪1.1.
      # Use the correct Athena database name and S3 output location.
      # 2. List available tables in the database.
      # 3. For the baseline timeseries table, print: column names, dtypes, and a
      ↪3-row sample.
      # 4. Identify and print: the bldg_id column, timestamp column, county
      ↪geography
      # column(s) (FIPS or GISJOIN), and total electricity consumption column(s).
      # 5. Store confirmed column names as constants for use in later steps.
      #
```

```

# INPUTS: AWS credentials (from environment). BuildStockQuery installed.
# OUTPUTS:
#   - `rsq` (BuildStockQuery): initialized query object
#   - `BLDG_ID_COL` (str): confirmed bldg_id column name
#   - `TIMESTAMP_COL` (str): confirmed timestamp/hour column name
#   - `COUNTY_COL` (str): confirmed county geography column name
#   - `ELEC_TOTAL_COL` (str): confirmed total electricity column name
# CONSTRAINTS: Type hints on any helper functions. Print schema clearly.
#               If a column name is uncertain, print all available columns and
↳ flag for review.

# if not bsq_available:
#     raise RuntimeError("BuildStockQuery not available - run Step 1 first.")

# def print_schema_summary(df: pd.DataFrame, label: str = "Sample") -> None:
#     """Print column names, dtypes, and a preview of a DataFrame.

#     Args:
#         df: DataFrame to summarize.
#         label: Descriptive label for the printout.
#     """
#     print(f"\n{'=' * 70}")
#     print(f" {label}")
#     print(f"{'=' * 70}")
#     print(f" Shape : {df.shape}")
#     print(f"\n Columns ({len(df.columns)}):")
#     for col in df.columns:
#         print(f"    {col:<60s} {df[col].dtype}")
#     print(f"\n First 3 rows:")
#     display(df.head(3))

# def identify_columns(columns: list[str]) -> dict[str, str | None]:
#     """Scan a column list and return best-match names for key fields.

#     Args:
#         columns: List of column name strings from an Athena table.

#     Returns:
#         Dict with keys 'bldg_id', 'timestamp', 'county', 'elec_total',
#         each mapped to the matched column name or None.
#     """
#     result: dict[str, str | None] = {
#         "bldg_id": None,
#         "timestamp": None,
#         "county": None,

```



```

#         "elec_total": None,
#     }
#     for col in columns:
#         cl = col.lower()
#         if cl in ("bldg_id", "building_id"):
#             result["bldg_id"] = col
#         elif cl in ("timestamp", "time", "hour", "datetime"):
#             result["timestamp"] = col
#         elif "county" in cl:
#             result["county"] = col
#         elif "electricity" in cl and "total" in cl and "energy_consumption"
#             in cl:
#             result["elec_total"] = col
#     return result

# # 1. Initialize BuildStockQuery for EUSS 2022.1.1
# #
# # Known parameters for EUSS 2022.1.1 (ResStock AMY2018, 550k sample, release
# # 1.1):
# # - Internal NREL Athena: db_name='euss-final',
# #   table_name='euss_res_final_2018_550k_20220901'
# # - OEDI published: db_schema='resstock_oedi', table names vary by user
# # setup.
# #
# # The code below attempts the OEDI configuration first, then falls back to the
# # internal configuration. Adjust workgroup / table names for your AWS account.
# #
# # Reference: https://github.com/NREL/buildstock-query/wiki/Getting-Started

# # ----- CONFIGURATION (edit these if your Athena setup differs) -----
# # ATHENA_WORKGROUP: str = "eulp" # Change to your Athena workgroup
# # EUSS_DB_NAME: str = "euss-final"
# # EUSS_TABLE_NAME: str = "euss_res_final_2018_550k_20220901"

# rsq = None

# try:
#     rsq = BuildStockQuery(
#         workgroup=ATHENA_WORKGROUP,
#         db_name=EUSS_DB_NAME,
#         table_name=EUSS_TABLE_NAME,
#         buildstock_type="resstock",
#         skip_reports=True,
#     )
#     print(f" BuildStockQuery initialized")
#     print(f" workgroup : {ATHENA_WORKGROUP}")

```

```

#     print(f" db_name      : {EUSS_DB_NAME}")
#     print(f" table_name   : {EUSS_TABLE_NAME}")
# except Exception as e:
#     print(
#         f" BuildStockQuery initialization failed: {e}\n\n"
#         f" Attempted: workgroup='{ATHENA_WORKGROUP}', "
#         f"db_name='{EUSS_DB_NAME}', table_name='{EUSS_TABLE_NAME}'\n\n"
#         f" Troubleshooting:\n"
#         f"     1. Open the AWS Athena console and verify the database & table
↪exist.\n"
#         f"     2. If using OEDI published data, pass db_schema='resstock_oedi'
↪and\n"
#         f"         supply a 3-tuple of (baseline_metadata, timeseries,
↪upgrade_metadata)\n"
#         f"         table names.\n"
#         f"     3. Check the BuildStockQuery wiki: "
#         f"https://github.com/NREL/buildstock-query/wiki/Getting-Started"
#     )
#     raise

# # 2. List available tables
# print(f"\n Baseline (annual) table : {rsq.bs_table}")
# print(f" Timeseries table          : {rsq.ts_table}")
# try:
#     upgrade_tables = rsq.up_table if hasattr(rsq, "up_table") else "N/A"
#     print(f" Upgrade table(s)          : {upgrade_tables}")
# except Exception:
#     pass

# # 3. Discover baseline timeseries schema
# # Pull a tiny sample (3 rows, upgrade_id=0 = baseline) to inspect column
↪names.
# print("\n Querying 3-row sample from baseline timeseries table... ")
# df_schema_sample = rsq.query(
#     enduses=["out.electricity.total.energy_consumption"],
#     group_by=["building_id", "county"],
#     upgrade_id=0,
#     annual_only=False,
#     limit=3,
#     get_query_only=False,
# )
# print_schema_summary(df_schema_sample, label="Baseline timeseries sample (3
↪rows)")

# # 4. Identify key column names
# detected = identify_columns(list(df_schema_sample.columns))

```

```

# BLDG_ID_COL: str = detected["bldg_id"] or "building_id"
# TIMESTAMP_COL: str = detected["timestamp"] or "timestamp"
# COUNTY_COL: str = detected["county"] or "in.county"
# ELEC_TOTAL_COL: str = detected["elec_total"] or "out.electricity.total.
    ↪energy_consumption"

# print(f"\n Confirmed column names ")
# print(f" BLDG_ID_COL      = '{BLDG_ID_COL}'")
# print(f"  TIMESTAMP_COL    = '{TIMESTAMP_COL}'")
# print(f"   COUNTY_COL       = '{COUNTY_COL}'")
# print(f"  ELEC_TOTAL_COL     = '{ELEC_TOTAL_COL}'")

# # Flag any unresolved columns
# for key, val in detected.items():
#     if val is None:
#         print(f"      '{key}' was not auto-detected - using fallback. "
#             f"Review column list above and update manually if needed.")

# print("\n Step 2 COMPLETE")

```

```
[35]: import boto3
```

```

[47]: # Check that boto3 can access AWS and list S3 buckets (sanity check for ↵
    ↪credentials and region)
BUCKET_NAME = input("Enter the bucket name (e.g., ↵
    ↪resstock-euss-query-results-{your-account-id}: ")

s3 = boto3.client("s3", region_name="us-west-2")
buckets = [bucket["Name"] for bucket in s3.list_buckets()["Buckets"]]
print(" Bucket found" if BUCKET_NAME in buckets else " Bucket not found - ↵
    ↪check S3 console")

```

Bucket found

```

[48]: # Check that the specified Athena workgroup exists (sanity check for Athena ↵
    ↪access and correct workgroup name)
WORKGROUP_NAME = input("Enter the Athena workgroup name (e.g., resstock-euss): ↵
    ↪")

athena = boto3.client("athena", region_name="us-west-2")
workgroup_names = [wg["Name"] for wg in athena.list_work_groups()["WorkGroups"]]
print(" Workgroup found" if WORKGROUP_NAME in workgroup_names else " Workgroup ↵
    ↪not found")

```

Workgroup found

```

[53]: # Check that the specified Glue database exists (sanity check for Glue access ↵
    ↪and correct database name)

```

```

DB_NAME = input("Enter the Glue database name (e.g., euss-oedi): ")

glue = boto3.client("glue", region_name="us-west-2")
db_names = [db["Name"] for db in glue.get_databases()["DatabaseList"]]
print(" Database found" if DB_NAME in db_names else " Database not found")

```

Database found

```

[54]: # Expected: two tables - timeseries and metadata
# Note the exact names - needed for the rename step in Part 5
glue = boto3.client("glue", region_name="us-west-2")
tables = glue.get_tables(DatabaseName=DB_NAME)["TableList"]
table_names = [t["Name"] for t in tables]
print(f"Tables in {DB_NAME}: {table_names}")

```

Tables in euss-oedi: ['resstock_amy2018_release_1_1_by_state',
'resstock_amy2018_release_1_1_metadata']

```

[56]: import boto3

glue = boto3.client("glue", region_name="us-west-2")

# Replace with actual name from Part 4
TABLE_TO_FIX = input(f"Enter the name of the table to fix (e.g., ↵
↳ resstock_amy2018_release_1_1_by_state): ").strip()

# Step 1: Fetch the full table definition before deleting
table = glue.get_table(DatabaseName=DB_NAME, Name=TABLE_TO_FIX)["Table"]

# Step 2: Delete the crawler-generated table
glue.delete_table(DatabaseName=DB_NAME, Name=TABLE_TO_FIX)
print(f" Deleted: {TABLE_TO_FIX}")

# Step 3: Fix the upgrade partition key type in our local copy
for key in table["PartitionKeys"]:
    if key["Name"] == "upgrade":
        key["Type"] = "int"
        print(f" Fixed: upgrade → int")

# Step 4: Recreate with corrected schema
glue.create_table(
    DatabaseName=DB_NAME,
    TableInput={
        "Name": table["Name"],
        "StorageDescriptor": table["StorageDescriptor"],
        "PartitionKeys": table["PartitionKeys"],
        "TableType": table.get("TableType", ""),
        "Parameters": table.get("Parameters", {}),
    },
)

```

```

    }
)
print(f" Recreated: {TABLE_TO_FIX} with upgrade as int")

# Step 5: Verify
updated = glue.get_table(DatabaseName=DB_NAME, Name=TABLE_TO_FIX)["Table"]
for key in updated["PartitionKeys"]:
    if key["Name"] == "upgrade":
        print(f" Confirmed upgrade type: {key['Type']}")

```

```

Deleted: resstock_amy2018_release_1_1_by_state
Fixed: upgrade → int
Recreated: resstock_amy2018_release_1_1_by_state with upgrade as int
Confirmed upgrade type: int

```

[]:

Step 3: County Geography Mapping — FIPS / GISJOIN to Shapefile

ResStock EUSS uses either FIPS codes or GISJOIN identifiers for county-level geography. We need to confirm which identifier is present and map it to standard Census TIGER/Line shapefiles for choropleth visualization.

GISJOIN format: G + state FIPS (2 digits, zero-padded) + 0 + county FIPS (3 digits, zero-padded). Example: Allegheny County PA = G4200030.

Standard FIPS format: 5-digit string. Example: Allegheny County PA = 42003.

Goal of this step: Build a lookup table `county_geo_df` that maps ResStock's county identifier → FIPS → county name → state, so all downstream results can be joined to shapefiles.

Test case: Confirm that Allegheny County, PA appears with FIPS 42003.

[]: *# CONTEXT: ResStock EUSS 2022.1.1 uses county-level geography identifiers (FIPS ↪ or ↪ GISJOIN) to tag each building. We need to map these to standard Census TIGER/ ↪ Line ↪ county shapefiles for visualization. Python 3.11, geopandas already imported.*

TASK:

- # 1. Query the ResStock metadata table (annual file or Athena) to get all ↪ ↪ unique ↪ county geography values from `COUNTY\_COL`.*
- # 2. Determine the format: is it FIPS (5-digit int or string) or GISJOIN ↪ ↪ (G+FIPS)?*
- # 3. Build a mapping DataFrame `county\_geo\_df` with columns: ↪ [resstock_county_id, fips_5digit, county_name, state_abbr]*
- # 4. Join to the Census TIGER/Line county shapefile at `SHAPEFILE\_PATH`.*

```
# 5. Confirm Allegheny County PA (FIPS 42003) is present. Print a sample of 5
  ↳ rows.
#
# INPUTS:
# - `rsq` (ResStockQuery): initialized BuildStockQuery object
# - `COUNTY_COL` (str): confirmed county column name from Step 2
# - `SHAPEFILE_PATH` (str): path to county-level shapefile (from existing
  ↳ constants)
# OUTPUTS:
# - `county_geo_df` (pd.DataFrame): mapping table [resstock_county_id → FIPS
  ↳ → name]
# - `gdf_counties` (gpd.GeoDataFrame): shapefile joined to county_geo_df
# CONSTRAINTS: Type hints. Fail with clear message if Allegheny County is
  ↳ missing.
#
# Do not assume GISJOIN vs FIPS - detect from the data.
# --- PLACEHOLDER: Opus/Copilot drafts implementation below ---
```

[]:

Step 4: Extract Adopter Building IDs from TARE Results

The TARE model assigns each building (`bldg_id`) to an adoption tier: - **Tier 1** — Private NPV positive (saves money without incentives) - **Tier 2** — Private NPV positive only with IRA rebates - **Tier 3** — Requires public benefits (social cost of carbon) to be justified - **Tier 4** — No adoption pathway under current conditions

For the **economically-constrained scenario**, adopters = Tier 1 + Tier 2. For the **100% adoption counterfactual**, adopters = all filtered building IDs.

This step extracts both sets of building IDs, organized by county (FIPS), for use in the timeseries queries in Steps 5–6.

Output structure:

```
adopter_ids_by_county = {
    "42003": {
        # FIPS for Allegheny County, PA
        "tier1": [101, 204, ...],
        "tier2": [305, 412, ...],
        "constrained": [101, 204, 305, 412, ...], # tier1 + tier2
        "all_filtered": [101, 204, ..., 99999], # 100% adoption
    }
}
```

[]: # CONTEXT: TARE model results are stored in DATAFRAMES_BY_MP (loaded in Step
 ↳ 0c).
 ↳
 ↳ # Each row is a building (bldg_id index) with columns for private NPV, tier
 ↳ assignment,

```

# county geography, and applicability. We need to extract adopter building IDs
↳grouped
# by county for both the constrained and 100% adoption scenarios.
#
# TASK:
# 1. For the selected MP (selected_mps[0]), access
↳DATAFRAMES_BY_MP[mp]['fixed_base'].
# 2. Identify the tier assignment column(s). Print available columns if
↳uncertain.
# 3. Extract two sets of bldg_ids per county:
# (a) `constrained`: Tier 1 + Tier 2 adopters
# (b) `all_filtered`: all buildings that passed the occupancy/SF/
↳applicable filters
# 4. Build `adopter_ids_by_county` dict keyed by FIPS (from county_geo_df
↳mapping).
# 5. Print a summary: total adopters vs total buildings for the test county
↳(FIPS 42003).
#
# INPUTS:
# - `DATAFRAMES_BY_MP` (Dict[int, Dict]): TARE results loaded in Step 0c
# - `selected_mps` (List[int]): MP numbers selected in Step 0b
# - county FIPS derived directly from `in.county` via GISJOIN conversion
# OUTPUTS:
# - `adopter_ids_by_county` (Dict[str, Dict[str, List[int]]]):
#   keyed by FIPS + {"tier1", "tier2", "constrained", "all_filtered"}
# - `primary_mp` (int): the single selected MP (selected_mps[0])
# CONSTRAINTS: Type hints on all helper functions. Google/NumPy docstrings.
#               Fail fast if tier column names are not found - print available
↳columns.

from cmu_tare_model.utils.column_names import create_adoption_col

# Configuration
primary_mp: int = selected_mps[0]
DISCOUNT_RATE_KEY: str = "fixed_base"
RCM_MODEL_KEY: str = "inmap"

# Tier value strings (must match determine_adoption_potential_sensitivity.py)
TIER_1_VALUE: str = "Tier 1: Feasible"
TIER_2_VALUE: str = "Tier 2: Feasible vs. Alternative"

def gisjoin_to_fips(gisjoin: str) -> str:
    """Convert a GISJOIN county identifier to a 5-digit FIPS code.

    GISJOIN format: G + 2-digit state FIPS + 0 + 3-digit county FIPS.

```

Example: 'G4200030' → '42003'.

Args:

gisjoin: GISJOIN string from the EUSS ``in.county`` column.

Returns:

5-digit county FIPS code as a string.

Raises:

*ValueError: If *gisjoin* is shorter than 7 characters.*

```
"""
if len(gisjoin) < 7:
    raise ValueError(
        f"GISJOIN string too short ({len(gisjoin)} chars): '{gisjoin}'. "
        f"Expected format 'G##0###' (7 chars).")
)
return gisjoin[1:3] + gisjoin[4:7]
```

```
def find_adoption_column(df: pd.DataFrame, mp: int, cost_scenario: str) -> str:
    """Locate the adoption-tier column in a TARE output DataFrame.
```

Builds the expected column name using ``create\_adoption\_col`` for the IRA-reference scenario with central SCC, InMAP, ACS, and the ``fixed\_base`` discount rate. Falls back to a fuzzy search if the exact column is absent.

Args:

df: TARE output DataFrame (one row per building).

mp: Measure-package number (e.g. 3 or 4).

cost_scenario: REMDB cost scenario key (e.g. ``v4MID``).

Returns:

The matched column name string.

Raises:

*KeyError: If no matching adoption column is found. The error message includes *all* column names that contain 'adoption' so the caller can diagnose the mismatch.*

```
"""
expected = create_adoption_col(
    scenario_prefix=f"iraRef_mp{mp}_",
    category="heating",
    column_type="adoption",
    cost_scenario=cost_scenario,
    method_suffix=f"_{DISCOUNT_RATE_KEY}",
    scc_assumption="central",
```



```

        rcm_model=RCM_MODEL_KEY,
        cr_function="acs",
    )
    if expected in df.columns:
        return expected

    # Fuzzy fallback: find any column containing 'adoption'
    adoption_candidates = [c for c in df.columns if "adoption" in c.lower()]
    if adoption_candidates:
        raise KeyError(
            f"Expected adoption column '{expected}' not found.\n"
            f"  Candidates containing 'adoption' ({len(adoption_candidates)}):
↪\n"
            + "\n".join(f"    • {c}" for c in adoption_candidates)
        )
    raise KeyError(
        f"Expected adoption column '{expected}' not found, "
        f"and no columns containing 'adoption' exist.\n"
        f"  Available columns ({len(df.columns)}):\n"
        + "\n".join(f"    • {c}" for c in sorted(df.columns))
    )

def extract_adopter_ids(
    df_tare: pd.DataFrame,
    adoption_col: str,
) -> dict[str, dict[str, list[int]]]:
    """Build per-county adopter ID dictionary from a TARE output DataFrame.

    For each county (identified via the ``in.county`` GISJOIN column),
    extracts building IDs for Tier 1, Tier 2, constrained (T1 + T2),
    and all filtered buildings.

    Args:
        df_tare: TARE output DataFrame with ``bldg_id`` as index,
            containing ``in.county`` and the specified *adoption_col*.
        adoption_col: Name of the adoption-tier column.

    Returns:
        Nested dict keyed by 5-digit FIPS string + sub-dict with keys
        ``'tier1'``, ``'tier2'``, ``'constrained'``, ``'all_filtered'``,
        each mapping to a list of ``bldg_id`` integers.

    Raises:
        KeyError: If ``in.county`` or *adoption_col* is missing from *df_tare*.
    """
    required_cols = {"in.county", adoption_col}

```

```

missing = required_cols - set(df_tare.columns)
if missing:
    raise KeyError(
        f"Missing column(s) {missing} in TARE DataFrame.\n"
        f" Available columns:\n"
        + "\n".join(f" • {c}" for c in sorted(df_tare.columns))
    )

# Derive FIPS from GISJOIN
df_work = df_tare[["in.county", adoption_col]].copy()
df_work["county_fips"] = df_work["in.county"].apply(gisjoin_to_fips)

result: dict[str, dict[str, list[int]]] = {}

for fips, grp in df_work.groupby("county_fips"):
    bldg_ids = grp.index.tolist()
    tier_vals = grp[adoption_col]

    tier1_ids = grp.index[tier_vals == TIER_1_VALUE].tolist()
    tier2_ids = grp.index[tier_vals == TIER_2_VALUE].tolist()

    result[str(fips)] = {
        "tier1": tier1_ids,
        "tier2": tier2_ids,
        "constrained": tier1_ids + tier2_ids,
        "all_filtered": bldg_ids,
    }

return result

# 1. Access the TARE DataFrame
print(f"Primary MP: {primary_mp}")
print(f"Discount rate key: {DISCOUNT_RATE_KEY}")
print(f"RCM model key: {RCM_MODEL_KEY}")

df_tare_nested = DATAFRAMES_BY_MP[primary_mp][DISCOUNT_RATE_KEY]

# Handle nesting: DATAFRAMES_BY_MP[mp][dr_key] may be {rcm_model: DataFrame}
if isinstance(df_tare_nested, dict):
    if RCM_MODEL_KEY not in df_tare_nested:
        raise KeyError(
            f"RCM model '{RCM_MODEL_KEY}' not found. "
            f"Available: {list(df_tare_nested.keys())}"
        )
    df_tare: pd.DataFrame = df_tare_nested[RCM_MODEL_KEY]

```

```

    print(f"    Accessed_
↳DATAFRAMES_BY_MP[{primary_mp}]['{DISCOUNT_RATE_KEY}']['{RCM_MODEL_KEY}']")
else:
    df_tare = df_tare_nested
    print(f"    Accessed DATAFRAMES_BY_MP[{primary_mp}]['{DISCOUNT_RATE_KEY}']_
↳(direct DataFrame)")

print(f"    Shape: {df_tare.shape}")
print(f"    Index name: {df_tare.index.name}")

# 2. Find the adoption column
# Try each cost scenario in priority order until one matches.
adoption_col: str | None = None
for cs in REMDB_COST_SCENARIO_KEYS:
    try:
        adoption_col = find_adoption_column(df_tare, primary_mp, cs)
        print(f"\n    Adoption column found (cost_scenario='{cs}'):\n_
↳{adoption_col}")
        break
    except KeyError:
        continue

if adoption_col is None:
    # Final attempt - raise with full diagnostics from the first cost scenario
    adoption_col = find_adoption_column(df_tare, primary_mp,
↳REMDB_COST_SCENARIO_KEYS[0])

# Print tier distribution
tier_counts = df_tare[adoption_col].value_counts()
print(f"\n    Tier distribution (MP{primary_mp}) ")
for tier_val, count in tier_counts.items():
    print(f"    {tier_val:<45s} {count:>7,d}")
print(f"    {'TOTAL':<45s} {tier_counts.sum():>7,d}")

# 3. Extract adopter IDs by county
adopter_ids_by_county: dict[str, dict[str, list[int]]] = extract_adopter_ids(
    df_tare, adoption_col
)

n_counties = len(adopter_ids_by_county)
total_constrained = sum(len(v["constrained"]) for v in adopter_ids_by_county.
↳values())
total_all = sum(len(v["all_filtered"]) for v in adopter_ids_by_county.values())
print(f"\n    Adopter summary ")
print(f"    Counties           : {n_counties:,d}")
print(f"    Constrained (T1+T2): {total_constrained:,d}")
print(f"    All filtered        : {total_all:,d}")

```

```
# 4. Test case: Allegheny County, PA (FIPS 42003)
TEST_FIPS = "42003"
if TEST_FIPS in adopter_ids_by_county:
    ac = adopter_ids_by_county[TEST_FIPS]
    print(f"\n Allegheny County (FIPS {TEST_FIPS}) ")
    print(f" Tier 1      : {len(ac['tier1']):,d}")
    print(f" Tier 2      : {len(ac['tier2']):,d}")
    print(f" Constrained : {len(ac['constrained']):,d}")
    print(f" All filtered : {len(ac['all_filtered']):,d}")
else:
    print(f"\n FIPS {TEST_FIPS} (Allegheny County, PA) not found in results.")
    sample_fips = list(adopter_ids_by_county.keys())[:10]
    print(f" Available FIPS (first 10): {sample_fips}")

print("\n Step 4 COMPLETE")
```

[]:

Step 5: Test Query — Baseline Timeseries for Allegheny County (FIPS 42003)

Before building the full pipeline, validate the query pattern on a single county. Allegheny County, PA (FIPS 42003) is the primary case study.

What this step produces: - `df_ts_baseline_allegheny`: shape (8760 × N_buildings) or pre-aggregated (8760,) containing hourly baseline (MP0) electricity consumption for all buildings in county

Design decision to resolve here: - Query strategy A: Pull all individual building timeseries → aggregate locally in pandas (higher Athena cost, more flexible locally) - Query strategy B: Push aggregation into Athena SQL → pull only the hourly sum (lower cost, less flexible for adopter/non-adopter split)

Recommendation: Start with Strategy A on Allegheny County only to validate the schema, then decide whether to push aggregation to Athena for the national loop.

[]: # CONTEXT: Testing BuildStockQuery against the OEDI EUSS 2022.1.1 timeseries
↳ Athena
table for a single county (Allegheny County, PA, FIPS 42003) before scaling.
We want hourly baseline (MP0) electricity consumption for all buildings in
↳ this county.

TASK:
1. Filter `all\_filtered` building IDs for FIPS 42003 from
↳ `adopter\_ids\_by\_county`.
2. Query the EUSS baseline timeseries table via BuildStockQuery for those
↳ bldg_ids.
Use `BLDG\_ID\_COL`, `TIMESTAMP\_COL`, `ELEC\_TOTAL\_COL` confirmed in Step 2.

```

# 3. Return a DataFrame `df_ts_baseline_allegheny` with columns:
#     [bldg_id, hour (1-8760), baseline_kwh].
# 4. Print: shape, memory usage, min/max hour, min/max kWh. Sample 5 rows.
# 5. Time the query and print elapsed seconds - we'll use this to estimate
    ↪ national cost.
#
# INPUTS:
# - `rsq` (ResStockQuery): initialized BuildStockQuery object
# - `adopter_ids_by_county` (dict): from Step 4
# - `BLDG_ID_COL`, `TIMESTAMP_COL`, `ELEC_TOTAL_COL` (str): from Step 2
# OUTPUTS:
# - `df_ts_baseline_allegheny` (pd.DataFrame): [bldg_id, hour, baseline_kwh]
# - `query_time_s` (float): elapsed query time in seconds
# CONSTRAINTS: Type hints. Wrap query in try/except - print Athena error
    ↪ clearly.
#
#           Do not pull more columns than needed (minimize scan cost).
#
# --- PLACEHOLDER: Opus/Copilot drafts implementation below ---

```

[]:

Step 6: Test Query — Upgrade Timeseries (MP3 or MP4) for Allegheny County

Query the post-retrofit timeseries for the same county and building IDs. The upgrade table contains electricity consumption after heat pump installation.

Note: Only buildings where `applicability == True` have valid upgrade data. The `adopter_ids_by_county["42003"]["all_filtered"]` list already respects this filter.

Key column to confirm: Does the upgrade timeseries table have the same schema as the baseline table, or are column names different? Resolve this here before building the aggregation logic in Step 7.

[]:

```

# CONTEXT: Querying the ResStock EUSS upgrade timeseries table (MP3 or MP4) for
# Allegheny County, PA (FIPS 42003). This is the post-retrofit electricity
    ↪ consumption
# for buildings that receive a heat pump under the selected measure package.
#
# TASK:
# 1. Identify the correct Athena table name for the upgrade scenario (MP3 or
    ↪ MP4).
#     This may differ from the baseline table name - check Athena schema if
    ↪ uncertain.
# 2. Query the upgrade timeseries for bldg_ids in
    ↪ `adopter_ids_by_county["42003"]["all_filtered"]`.
# 3. Return `df_ts_upgrade_allegheny` with columns: [bldg_id, hour,
    ↪ retrofit_kwh].

```

```

# 4. Confirm schema matches baseline (same hour range, same bldg_id format).
# 5. Print shape, sample rows, and flag any bldg_ids present in baseline but
↳missing
# from upgrade (these are non-applicable buildings - should be zero after
↳filter).
#
# INPUTS:
# - `rsq` (ResStockQuery): initialized BuildStockQuery object
# - `primary_mp` (int): selected measure package (3 or 4)
# - `adopter_ids_by_county` (dict): from Step 4
# - `BLDG_ID_COL`, `TIMESTAMP_COL`, `ELEC_TOTAL_COL` (str): from Step 2
# OUTPUTS:
# - `df_ts_upgrade_alleggheny` (pd.DataFrame): [bldg_id, hour, retrofit_kwh]
# CONSTRAINTS: Type hints. Print a warning if any adopter bldg_ids are missing
# from the upgrade table - this indicates a data join issue.

# --- PLACEHOLDER: Opus/Copilot drafts implementation below ---

```

[]:

Step 7: Compute Scenario Demand Profile — Allegheny County (Test Case)

Applies the adopter/non-adopter mask to produce a scenario hourly demand profile.

Logic (per building, per hour): - If bldg_id is in adopter_ids → use retrofit_kwh (post-HP electricity) - Otherwise → use baseline_kwh (existing equipment electricity)

Two profiles produced: 1. **100% adoption:** all filtered buildings adopt → all use retrofit_kwh
2. **Constrained adoption:** Tier 1 + Tier 2 adopt; Tier 3 + Tier 4 use baseline

Peak load change = max(scenario_profile_kw) − max(baseline_profile_kw)

Units note: BuildStockQuery returns kWh per hour. Since each interval is 1 hour, kWh = kW for that hour. Multiply by sampling weight (~240) to convert from simulated units to real-world MW equivalents.

[]: # CONTEXT: Computing county-level scenario demand profiles for Allegheny
↳County, PA.
We have baseline and upgrade timeseries DataFrames from Steps 5-6.
The adopter mask determines which buildings use baseline vs retrofit
↳consumption.

TASK: Implement `compute\_county\_scenario\_profile()` that:
1. Merges `df\_ts\_baseline\_alleggheny` and `df\_ts\_upgrade\_alleggheny` on
↳[bldg_id, hour].
2. Accepts `adopter\_bldg\_ids` (List[int]) as parameter.
3. For each building-hour: assigns retrofit_kwh if adopter, else
↳baseline_kwh.

```

# 4. Aggregates across all buildings → hourly scenario demand profile (8760
→ values).
# 5. Applies sampling weight to convert to MW (confirm weight value from EUSS
→ metadata).
# 6. Returns a DataFrame with columns: [hour, baseline_mw, scenario_mw,
→ delta_mw].
#
# Then call this function twice:
# (a) adopter_bldg_ids = adopter_ids_by_county["42003"]["all_filtered"]
→ (100% adoption)
# (b) adopter_bldg_ids = adopter_ids_by_county["42003"]["constrained"]
→ (Tier 1+2 only)
#
# Print peak hour and peak load change for both scenarios.
#
# INPUTS:
# - `df_ts_baseline_allegheeny` (pd.DataFrame): [bldg_id, hour, baseline_kwh]
# - `df_ts_upgrade_allegheeny` (pd.DataFrame): [bldg_id, hour, retrofit_kwh]
# - `adopter_ids_by_county` (dict): from Step 4
# OUTPUTS:
# - `df_profile_100pct` (pd.DataFrame): [hour, baseline_mw, scenario_mw,
→ delta_mw]
# - `df_profile_constrained` (pd.DataFrame): same schema, Tier 1+2 adopters
→ only
# - `peak_results_allegheeny` (dict): peak hour, baseline peak MW, scenario
→ peak MW, delta MW
# CONSTRAINTS: Type hints required. Google/NumPy docstring on the function.
# Verify 8760 rows in output - raise ValueError if not.

def compute_county_scenario_profile(
    df_baseline: "pd.DataFrame",
    df_upgrade: "pd.DataFrame",
    adopter_bldg_ids: "list[int]",
    sampling_weight: float = 240.0,
) -> "pd.DataFrame":
    """
    # --- PLACEHOLDER: Opus/Copilot completes this function ---
    """
    raise NotImplementedError("Step 7 placeholder - implement with Copilot/
→ Opus")

```

[]:

Step 8: Validate — Compare Aggregated Profile Against EUSS Peak Load Columns

EUSS annual/metadata files include individual household peak load values (kW). These provide a

within-dataset sanity check before comparing to external benchmarks.

Validation approach: - Sum the EUSS individual peak load values for Allegheny County buildings → county peak (naive sum) - Compare to the profile-derived peak from Step 7 (baseline, 100% adoption) - These will not be identical (naive sum coincident peak), but should be same order of magnitude

External benchmark (after internal check passes): - Compare PA statewide peak load change to Maxim et al. (2024) 2018 vs. 2050 scenarios - Reference OEDI URL from notebook stub: https://data.openei.org/s3_viewer?bucket=oedi-data-lake&prefix=nrel-pds-building-stock%2Fend-u

Flag: If profile-derived peak differs from EUSS column sum by more than 20%, investigate before scaling to national loop.

```
[ ]: # CONTEXT: Validating our aggregated timeseries-derived peak load against the EUSS
      # annual metadata file's built-in peak load columns for Allegheny County, PA.
      # df_baseline (the annual/metadata CSV) is already loaded from Step 1 of the notebook.
      #
      # TASK:
      # 1. Identify the peak load column(s) in `df_baseline` (annual CSV).
      #    Look for columns containing 'peak' or 'max' in the column name - print candidates.
      # 2. Filter df_baseline to Allegheny County buildings (use county_geo_df mapping).
      # 3. Sum the individual peak load values → `naive_county_peak_kw`.
      #    (Naive sum is an upper bound - assumes all buildings peak simultaneously.)
      # 4. Compare to `peak_results_allegheny["baseline_peak_mw"] * 1000` (convert to kW).
      # 5. Compute ratio: naive_sum / profile_derived_peak. Print result with interpretation.
      # 6. Flag with a warning if ratio is outside [0.8, 5.0] (rough sanity bounds).
      #
      # INPUTS:
      # - `df_baseline` (pd.DataFrame): EUSS annual metadata CSV, loaded in Step 1
      # - `county_geo_df` (pd.DataFrame): county mapping from Step 3
      # - `peak_results_allegheny` (dict): peak load results from Step 7
      # OUTPUTS: Printed validation summary. No new DataFrames required.
      # CONSTRAINTS: Type hints on any helper. If peak column name is uncertain,
      #                print all column names containing 'peak', 'max', or 'load'.
      #
      # --- PLACEHOLDER: Opus/Copilot drafts implementation below ---
```

[]:

Step 9: National Loop — County-Level Peak Load Table

Run Step 7 validation on Allegheny County BEFORE running this step.

Scales the Allegheny County pipeline across all counties present in `adopter_ids_by_county`.

Design decisions to resolve before running: 1. **Query batching:** Query all buildings for a county in one Athena call, or batch by state first (reduces number of Athena connections)? 2. **Aggregation location:** Push the hourly sum to Athena SQL (cheaper, faster) or pull building-level data and aggregate in pandas (more flexible for adopter mask)? 3. **Checkpoint saving:** Save intermediate results per state to disk in case the loop fails mid-run (strongly recommended for a national run).

Expected output: `df_peak_results_national` with one row per county: `[fips, county_name, state, n_adopters_constrained, n_all_filtered, baseline_peak_mw, scenario_100pct_peak_mw, scenario_constrained_peak_mw, delta_100pct_mw, delta_constrained_mw, peak_hour_100pct, peak_hour_constrained]`

```
[ ]: # CONTEXT: Scaling the county-level peak load pipeline (Steps 5-7) to all
      ↪ counties
      # in the TARE model results. ResStock EUSS 2022.1.1 via AWS Athena /
      ↪ BuildStockQuery.
      # Python 3.11, conda env cmu-tare-model. Allegheny County (FIPS 42003)
      ↪ validated in Step 7.
      #
      # TASK: Implement `run_national_peak_load_loop()` that:
      #   1. Iterates over all FIPS keys in `adopter_ids_by_county`.
      #   2. For each county: queries baseline + upgrade timeseries (reuse logic from
      ↪ Steps 5-6),
      #       calls `compute_county_scenario_profile()` for both adoption scenarios.
      #   3. Stores peak load results in a running list + converts to DataFrame at
      ↪ end.
      #   4. Saves a checkpoint file per state (e.g., `peak_results_PA.csv`) so
      ↪ progress is
      #       not lost if the loop fails.
      #   5. Prints progress: "County X of N | FIPS {fips} | {county_name}, {state} |
      ↪ "
      #   6. Returns `df_peak_results_national` (pd.DataFrame) with schema described
      ↪ above.
      #
      # INPUTS:
      #   - `rsq` (ResStockQuery): initialized BuildStockQuery object
      #   - `adopter_ids_by_county` (dict): from Step 4
      #   - `county_geo_df` (pd.DataFrame): from Step 3
      #   - `primary_mp` (int): selected measure package
      #   - `PROJECT_ROOT` (str): from config, for checkpoint file paths
      # OUTPUTS:
      #   - `df_peak_results_national` (pd.DataFrame): county-level peak load results
      # CONSTRAINTS: Type hints. Google/NumPy docstring. Wrap each county query in
      ↪ try/except
```

```

#             - log failures to a `failed_counties` list and continue.
#             Do not run without Step 7 validation passing first.

def run_national_peak_load_loop(
    rsq: object,
    adopter_ids_by_county: "dict[str, dict[str, list[int]]]",
    county_geo_df: "pd.DataFrame",
    primary_mp: int,
    project_root: str,
    sampling_weight: float = 240.0,
) -> "pd.DataFrame":
    """
    # --- PLACEHOLDER: Opus/Copilot completes this function ---
    """
    raise NotImplementedError("Step 9 placeholder - implement with Copilot/
↪Opus")

```

```
[ ]:
```

Step 10: Export Results for Paper Figures

Exports the national county-level peak load table for use in: - **Figure XX (paper Section 3.6):** County choropleth of peak load change under 100% adoption and economically-constrained adoption, by technology scenario (MP3/MP4) - **Allegheny County case study panel:** Bar chart comparing baseline vs scenario peak across all technology scenarios

Files exported: - `peak_load_results_MP{mp}_national.csv` — full national county table - `peak_load_results_MP{mp}_allegheny.csv` — Allegheny County only (for case study)

```

[ ]: # CONTEXT: Exporting national county-level peak load results for paper figures.
# Results are in `df_peak_results_national` from Step 9.
#
# TASK:
# 1. Define OUTPUT_DIR using PROJECT_ROOT (create directory if it doesn't
↪exist).
# 2. Save `df_peak_results_national` as CSV with filename including MP number
↪and date.
# 3. Filter to Allegheny County (FIPS 42003) → save as separate CSV.
# 4. Print file paths and row counts for both exports.
# 5. Print a summary table: top 10 counties by peak load delta (100% adoption
↪scenario).
#
# INPUTS:
# - `df_peak_results_national` (pd.DataFrame): from Step 9
# - `primary_mp` (int): for filename
# - `PROJECT_ROOT` (str): from config
# OUTPUTS: Two CSV files written to disk. Printed confirmation.

```

```
# CONSTRAINTS: Use pathlib.Path throughout. Include ISO date in filename.
#               Do not overwrite existing files - append a suffix if file exists.

# --- PLACEHOLDER: Opus/Copilot drafts implementation below ---
```

```
[ ]:
```

Handoff Notes for Claude Opus

What is complete (do not modify): - Steps 0–0c: TARE data loading, MP selection — working, tested - Step 1: EUSS annual CSV loading + 3-stage filter — working, tested

What needs implementation (in order): 1. Step 1 (NEW): BuildStockQuery install + AWS credential check 2. Step 2: OEDI Athena schema discovery — **must resolve column names before any other step** 3. Step 3: County geography mapping (FIPS/GISJOIN → shapefile) 4. Step 4: Adopter ID extraction from TARE results 5. Steps 5–6: Single-county test queries (Allegheny County first, always) 6. Step 7: Scenario demand profile + peak calculation 7. Step 8: Validation against EUSS built-in peak values 8. Step 9: National loop (only after Step 8 passes) 9. Step 10: Export

Critical constraint: ResStock 2022.1.1 (EUSS) only. Do not reference 2025.1.

Primary test case: Allegheny County, PA — FIPS 42003.

Code standards: Type hints on all functions. Google/NumPy docstrings. Fail fast.

If Athena table names are unknown: Check the OEDI S3 browser first: https://data.openei.org/s3_viewer?bucket=oedi-data-lake&prefix=nrel-pds-building-stock%2Fend-u. Then check the BuildStockQuery wiki for the correct initialization parameters.

```
[ ]:
```

Geospatial Visualization

```
[ ]: gdf_conus = None
      gdf_alaska = None

      try:
          gdf_states_raw = gpd.read_file(SHAPEFILE_PATH)
          _, gdf_conus, gdf_alaska = prepare_state_geodataframe(gdf_states_raw,
↳df_spark, merge_col='state')
          print(f" Geodataframe prepared: CONUS={len(gdf_conus)},
↳AK={len(gdf_alaska)}")
      except Exception as e:
          print(f" Shapefile not loaded: {e} - skipping maps")
```

```
[ ]: # Demand change map (diverging)
      if gdf_conus is not None and gdf_alaska is not None:
```

```

_, gdf_demand_conus, gdf_demand_alaska = prepare_state_geodataframe(
    gdf_states_raw, df_demand_state, merge_col='state'
)
create_choropleth_map(
    gdf_demand_conus, gdf_demand_alaska,
    column='elec_change_gwh',
    title='Electricity Demand Change Under 100% HP Adoption by State_
↪(2022)',
    cbar_label='Electricity Demand Change (GWh)\n(positive = more grid_
↪electricity needed)',
    output_path=os.path.join(PROJECT_ROOT,
↪"state_elec_demand_change_map_2022.png"),
    cmap='coolwarm', show_plot=True,
)
print(" Demand map generated")
else:
    print(" Maps skipped")

```

Display Results

```

[ ]: # =====
# DISPLAY: DEMAND CHANGE
# =====

# print(f"\n===== DEMAND CHANGE (MP{primary_mp}, GWh, all fuels, 100% adoption)_
↪=====\\n")
# display(df_demand_state[['state', 'home_count', 'elec_change_gwh',
#                           'pct_elec_demand_change', 'site_energy_change_gwh',
#                           'pct_site_energy_change']])

# print(f"\n DISPLAY COMPLETE")

```